

unordered_set 정리

정렬하지 않고 해시 기반으로 중복 없는 값을 빠르게 저장하는 컨테이너입니다.

사용처 (Where)

- 정렬이 필요 없고 존재 여부만 빠르게 확인할 때
- 방문 체크
- 중복 제거를 빠르게 하고 싶을 때

방법 (How to Use)

- insert로 추가
- find/count로 존재 확인
- 순회 순서는 예측하지 않음
- 사용자 정의 타입은 해시 함수가 필요할 수 있음

기본 정보

- 필요 헤더: `#include <unordered_set>`
- 기본 선언: `unordered_set<int> s;`

왜 써야 할까? (Why)

- set보다 평균 검색이 빠름
- 방문 여부 체크 코드를 간단하게 만들
- 정렬이 필요 없는 중복 제거에 적합

주요 함수

함수/문법	의미
<code>insert(x)</code>	값 추가
<code>find(x)</code>	검색
<code>count(x)</code>	존재 여부
<code>erase(x)</code>	삭제
<code>size()</code>	개수

기본 코드 예시

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main() {
    unordered_set<int> s = {3, 1, 3, 2};

    if (s.count(2)) cout << "exists\n";
}
```

unordered_set 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
insert/find/erase	평균 $O(1)$
최악	$O(n)$
정렬 순회	지원 안 함

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	이미 구현된 기능 사용	자료구조 설계와 예외 처리 필요	배열/포인터 직접 관리
코드 길이	짧고 읽기 쉬움	길고 버그 가능성 증가	간단하지만 확장성이 낮음
성능	일반 상황에서 충분히 최적화	구현 실력에 따라 달라짐	고정 크기에서는 단순함
안정성	범위/메모리 관리 부담 감소	메모리/인덱스 실수 가능	범위 초과 위험 큼
추천 상황	대부분의 C++ 코드	자료구조 원리 학습	아주 단순한 고정 크기 데이터

응용: 이미 나온 값인지 확인

입력된 값이 처음 등장하는 값인지 unordered_set으로 확인합니다.

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main() {
    int arr[] = {4, 2, 4, 7};
    unordered_set<int> seen;

    for (int x : arr) {
        if (seen.count(x)) cout << x << " repeated\n";
        seen.insert(x);
    }
}
```

처음 배울 때 자주 하는 실수

- 출력 순서가 정렬되어 있지 않음
- 최악의 경우 $O(n)$ 이 될 수 있음
- 사용자 정의 타입은 별도 hash가 필요

비슷한 항목과 차이

- set은 자동 정렬과 $O(\log n)$ 검색
- unordered_map은 key-value 저장
- vector는 중복 제거 전 정렬이 필요할 수 있음

한 줄 정리: unordered_set은(는) '정렬이 필요 없고 존재 여부만 빠르게 확인할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.